



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт искусственного интеллекта (ИИИ)

ОТЧЕТ ПО ИТОГОВОМУ ПРОЕКТУ

по программе ДПО

«Инженерия искусственного интеллекта»

на тему:

«Сервис прогнозирования риска долгого закрытия задач в Jira»

Студент группы ИНБО-10-22

«__» _____ 2026 г.

Н.В. Чайка

(подпись и расшифровка подписи)

Руководитель работы

Ш.Г. Магомедов

(подпись и расшифровка подписи)

Москва 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 ПОСТАНОВКА ЗАДАЧИ И АНАЛИЗ ДАННЫХ.....	5
1.1 Постановка задачи	5
1.2 Источник и описание данных	7
1.3 Разведочный анализ данных	8
2 МОДЕЛИ И ЭКСПЕРИМЕНТЫ	13
2.1 Базовые модели	13
2.2 Улучшенные модели и эксперименты	14
2.3 Экспериментальный протокол и сравнение моделей	15
2.4 Выбор финальной модели.....	18
3 АРХИТЕКТУРА РЕШЕНИЯ И СЕРВИС.....	21
3.1 Архитектура пайплайна.....	21
3.2 API и эндпоинты	23
3.3 Технологический стек и контейнеризация.....	25
4 НАБЛЮДАЕМОСТЬ, КОНФИГУРАЦИЯ И БЕЗОПАСНОСТЬ	27
4.1 Логирование и наблюдаемость.....	27
4.2 Конфигурация.....	29
4.3 Работа с секретами и данными	30
ЗАКЛЮЧЕНИЕ	32
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	35
ПРИЛОЖЕНИЕ А	36

ВВЕДЕНИЕ

Актуальность проекта заключается в разработке инструмента раннего выявления Jira issues с повышенным риском долгого закрытия. Такой сервис не заменяет принятие управленческих решений, но может использоваться как вспомогательный механизм при первичной triage-оценке задач. Модель позволяет заранее выделить потенциально проблемные обращения, после чего ответственный специалист может уточнить требования, повысить приоритет, проверить заполнение компонента, перераспределить ресурсы или взять задачу на дополнительный контроль.

Целью итогового проекта является разработка воспроизводимого ML-сервиса для прогнозирования риска долгого закрытия Jira issue на основе признаков, доступных на раннем этапе обработки задачи.

Для достижения цели были поставлены следующие задачи:

1. Выполнить анализ исходных Jira-данных, определить структуру датасета, качество заполнения полей и возможность формирования целевой переменной.
2. Подготовить признаки и целевую переменную для ML-задачи бинарной классификации, исключив поля, приводящие к утечке данных.
3. Обучить baseline и улучшенные модели машинного обучения, сравнить их по целевым метрикам и выбрать финальную модель.
4. Реализовать сервис прогнозирования на FastAPI с эндпоинтами /health и /predict, Swagger UI и примером запроса.
5. Подготовить инженерную часть проекта: конфигурацию, логирование, тесты, Dockerfile, .env.example, README и сценарий демонстрации.

В прикладном смысле проект решает задачу раннего предупреждения о риске долгого закрытия Jira issue. С точки зрения машинного обучения задача относится к бинарной классификации: сервис получает признаки issue и возвращает прогноз, будет ли задача относиться к группе долгого закрытия. Основными пользователями сервиса могут быть project manager, тимлид, аналитик сопровождения или сотрудник команды поддержки. Типовой сценарий использования: на этапе первичной triage-оценки специалист отправляет признаки issue в сервис, получает вероятность риска и принимает решение, требуется ли дополнительное внимание к задаче.

В результате проект охватывает полный минимальный жизненный цикл инженерии искусственного интеллекта: получение и анализ данных, предобработку, обучение моделей, сохранение финального артефакта, разработку API-сервиса, проверку работоспособности, контейнеризацию и подготовку проекта к демонстрации.

1 ПОСТАНОВКА ЗАДАЧИ И АНАЛИЗ ДАННЫХ

1.1 Постановка задачи

При постановке задачи учитывались несколько ограничений. Во-первых, сервис должен работать достаточно быстро, так как прогноз предполагается использовать на этапе первичной оценки задачи, а не в длительном офлайн-анализе. Во-вторых, модель должна быть достаточно интерпретируемой на уровне общего понимания факторов риска: важно понимать, какие признаки используются и почему они доступны до закрытия issue. В-третьих, качество модели должно оцениваться не только по общей доле правильных ответов, но и по способности находить положительный класс, то есть задачи с долгим закрытием. В-четвертых, решение должно быть воспроизводимым и не требовать тяжелой инфраструктуры, поэтому в проекте используются классические модели scikit-learn и FastAPI-сервис.

Бизнес-задача проекта состоит в раннем выявлении Jira issues, которые с высокой вероятностью будут закрываться дольше обычного. С практической точки зрения это может помочь project manager, тимлиду или специалисту поддержки обратить внимание на потенциально проблемные задачи до того, как они уже накопят значительное время ожидания.

В рамках проекта задача формализована как задача бинарной классификации. Для каждой issue модель должна определить, относится ли она к классу задач с долгим закрытием. Входными данными являются признаки задачи, которые могут быть доступны на этапе первичной triage-оценки: тип задачи, приоритет, наличие приоритета, наличие компонента, длина краткого описания, число слов в кратком описании, длина полного описания и число слов в полном описании.

Выходом модели является бинарный прогноз:

`is_delayed = 1` — задача относится к классу долгого закрытия;

`is_delayed = 0` — задача не относится к классу долгого закрытия.

Дополнительно сервис возвращает вероятность принадлежности к классу долгого закрытия. Это важно, потому что в практическом сценарии пользователю полезен не только жесткий класс, но и степень риска. Например, `issue` с вероятностью 0.15 и `issue` с вероятностью 0.80 требуют разного уровня внимания, даже если итоговый бинарный прогноз может быть одинаковым при выбранном пороге.

Целевая переменная строится на основе реальных дат создания и закрытия задачи. Для каждой решенной `issue` рассчитывается время закрытия:

`resolution_time_days = Resolved - Created.`

`Issue` считается долгой, если время ее закрытия превышает 75-й перцентиль по валидным решенным задачам. В текущей версии проекта этот порог составляет 305.2056 дня. Таким образом, положительный класс соответствует верхней четверти задач по длительности закрытия.

Выбор 75-го перцентиль позволяет сформулировать задачу не как прогноз точной даты закрытия, а как выявление относительно долгих и потенциально проблемных `issues`. Такой подход удобен для прикладного сценария раннего предупреждения: команде важно не столько предсказать точное количество дней до закрытия, сколько заранее понять, какие задачи могут попасть в группу риска.

Для оценки качества использовались Accuracy, Precision, Recall, F1 и ROC-AUC. Accuracy показывает общую долю правильных ответов, но не является основной метрикой из-за дисбаланса классов. Precision показывает, насколько точны предупреждения модели о долгом закрытии. Recall отражает, какую долю действительно долгих задач модель смогла обнаружить. F1 используется как баланс между Precision и Recall. ROC-AUC показывает способность модели ранжировать `issues` по вероятности риска независимо от конкретного порога классификации.

При построении признаков отдельно учитывалась проблема утечки данных. В модель не включались поля `Status`, `Resolution`, `Resolved`, `Updated` и `resolution_time_days`, так как они содержат информацию, которая становится

известна уже после закрытия или изменения состояния задачи. Использование таких признаков привело бы к завышенным метрикам и некорректной оценке качества модели. Также из финального набора признаков были исключены календарные признаки создания, поскольку в датасете был выявлен заметный временной сдвиг.

1.2 Источник и описание данных

Источник данных — открытый датасет Jira Dataset, опубликованный на Kaggle [1]. Локальный путь к исходному файлу в проекте: `project/data/jira_dataset.csv`. Инструкция по получению данных находится в `project/data/README.md`. Так как исходный датасет размещен на Kaggle, CSV-файл не должен коммититься в публичный репозиторий без проверки условий использования.

Данные представлены в табличном формате CSV. В исходной таблице содержатся поля разных типов: категориальные признаки, например тип задачи и приоритет; текстовые признаки, например `summary` и `description`; календарные признаки, например `Created` и `Resolved`; а также служебные поля состояния задачи. В финальную модель включены только те признаки, которые могут быть доступны до закрытия `issue`.

Для демонстрационной проверки в папке `project/data/` размещена малая обезличенная выборка `project/data/demo_sample.csv`. Она повторяет структуру ключевых входных полей и позволяет проверить формат данных без публикации полного исходного CSV-файла. Для проверки API дополнительно используется демонстрационный запрос `project/src/demo_request.json`.

Исходный датасет представляет собой выгрузку Jira issues и содержит 49000 строк. В таблице присутствуют поля, описывающие задачу, ее тип, приоритет, компонент, текстовое описание, дату создания, дату закрытия и другие атрибуты. Для целей обучения используются только те issues, у которых

доступны валидные даты Created и Resolved. После фильтрации таких решенных задач остается 16562 строки.

На основе исходных данных формируется подготовленная таблица, которая сохраняется в `project/artifacts/jira_issues_prepared.csv`. Она содержит рассчитанную целевую переменную `is_delayed` и финальный набор признаков, используемых при обучении модели.

Финальные признаки модели включают:

`issue_type` — тип задачи, например Bug или Suggestion;

`priority` — приоритет задачи;

`has_priority` — бинарный признак наличия заполненного приоритета;

`component_present` — бинарный признак наличия указанного компонента;

`summary_length` — длина краткого описания задачи;

`summary_word_count` — количество слов в кратком описании;

`description_length` — длина полного описания задачи;

`description_word_count` — количество слов в полном описании.

Выбранные признаки соответствуют сценарию раннего применения модели. Они могут быть доступны до закрытия задачи и не содержат прямой информации о будущем результате. Это позволяет использовать модель как вспомогательный инструмент при первичной обработке `issue`, а не как ретроспективный анализ уже закрытых задач.

1.3 Разведочный анализ данных

Разведочный анализ данных выполнен в `notebook project/notebooks/01_eda.ipynb`. Ноутбук воспроизводимо загружает исходный CSV-файл, проверяет структуру таблицы, типы признаков, пропуски, базовую статистику, рассчитывает целевую переменную и строит графики через `matplotlib`. Важно, что `notebook` не просто отображает заранее подготовленные

изображения, а самостоятельно выполняет основные шаги загрузки, предобработки и визуализации данных.

В ходе EDA была проведена проверка заполненности ключевых полей. Поле Resolved заполнено примерно у 33.8% исходных строк. Это означает, что значительная часть задач в исходном датасете не имеет даты закрытия и не может быть напрямую использована для обучения модели, где целевая переменная строится на основе времени закрытия. Поэтому для обучения были оставлены только issues с валидными датами Created и Resolved.

После фильтрации решенных задач было рассчитано время закрытия resolution_time_days. Распределение этой величины имеет выраженный длинный хвост: большинство задач закрывается сравнительно быстрее, однако часть issues имеет очень большое время закрытия, максимум превышает 2500 дней. Для более читаемой визуализации график времени закрытия в отчете ограничен по оси, но сам факт длинного хвоста учитывается при анализе.

Целевая переменная is_delayed строится по 75-му процентилю времени закрытия. В результате положительный класс составляет 24.85% от валидных решенных задач. Это указывает на умеренный дисбаланс классов: обычных задач существенно больше, чем задач с долгим закрытием. Данный факт повлиял на выбор метрик и интерпретацию результатов моделей.

Также в ходе EDA было выявлено, что все валидные решенные issues относятся к одному проекту SRCTREEWIN. По этой причине поле Project key не используется как информативный признак в финальной модели: оно не помогает различать задачи внутри подготовленной выборки. Кроме того, этот факт является важным ограничением проекта, поскольку модель обучается на данных одного Jira-проекта и требует дополнительной проверки перед переносом на другие проекты или организации.

Предобработка данных вынесена в отдельный модуль project/src/preprocessing.py. В проекте используется единый preprocessing pipeline, который применяется как на этапе обучения, так и на этапе предсказания. Для числовых признаков предусмотрены заполнение пропусков и

масштабирование, для категориальных признаков — заполнение пропусков и one-hot encoding. Бинарные признаки приводятся к числовому виду. Такой подход снижает риск расхождения между обучением и эксплуатацией модели.

На этапе предобработки выполнялись очистка и приведение данных к формату, пригодному для обучения модели. Даты Created и Resolved преобразовывались к datetime-формату, строки без валидного времени закрытия исключались из обучающей выборки, после чего рассчитывалось resolution_time_days. Для текстовых полей не применялась полноценная NLP-токенизация или embeddings; вместо этого из summary и description извлекались простые числовые признаки: длина текста и количество слов. Категориальные признаки обрабатывались через заполнение пропусков и OneHotEncoder, числовые признаки — через заполнение пропусков и StandardScaler.

Ключевые результаты EDA можно сформулировать следующим образом: исходный датасет содержит достаточное количество задач для построения модели, однако обучение возможно только на подмножестве решенных issues; целевая переменная имеет умеренный дисбаланс; время закрытия обладает длинным хвостом; признаки должны выбираться с учетом риска утечки данных; переносимость модели на другие Jira-проекты требует дополнительной проверки.

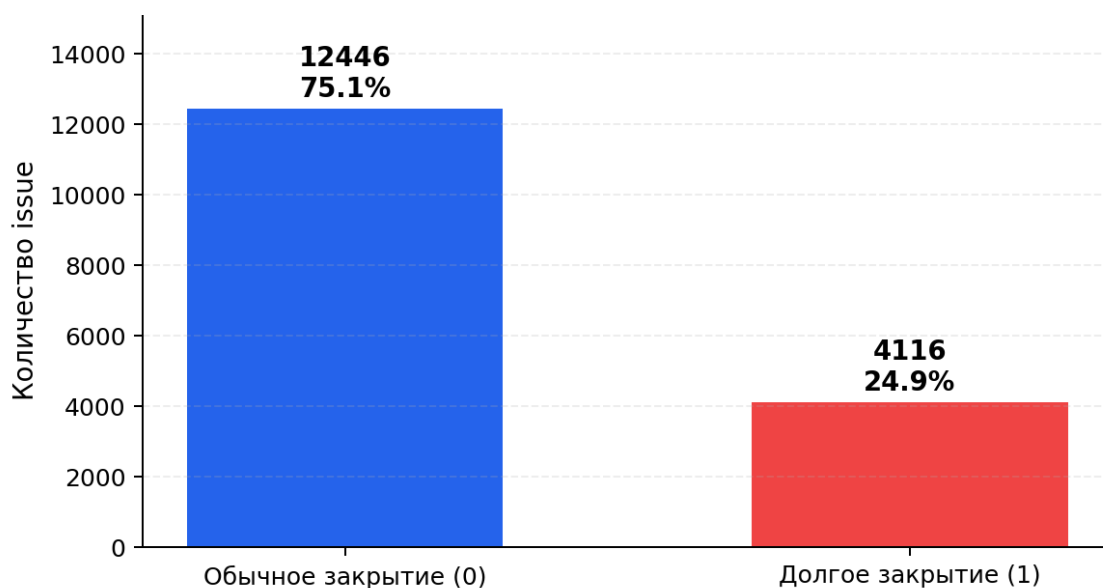


Рисунок 1.1 — распределение целевой переменной is_delayed из EDA-ноутбука

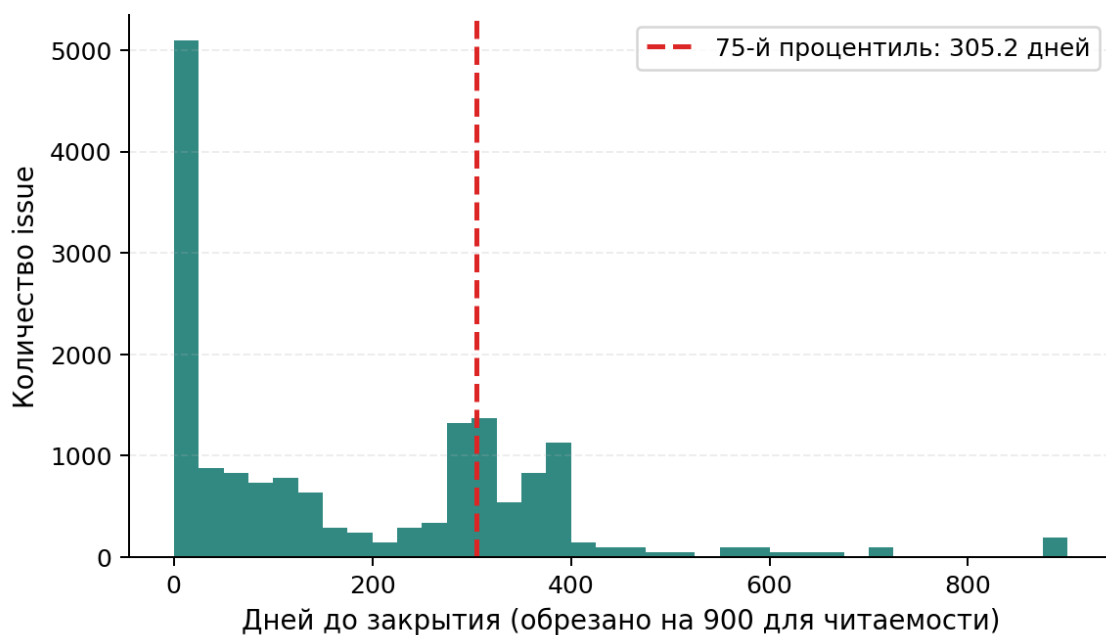


Рисунок 1.2 — Распределение времени закрытия Jira issue из EDA-ноутбука

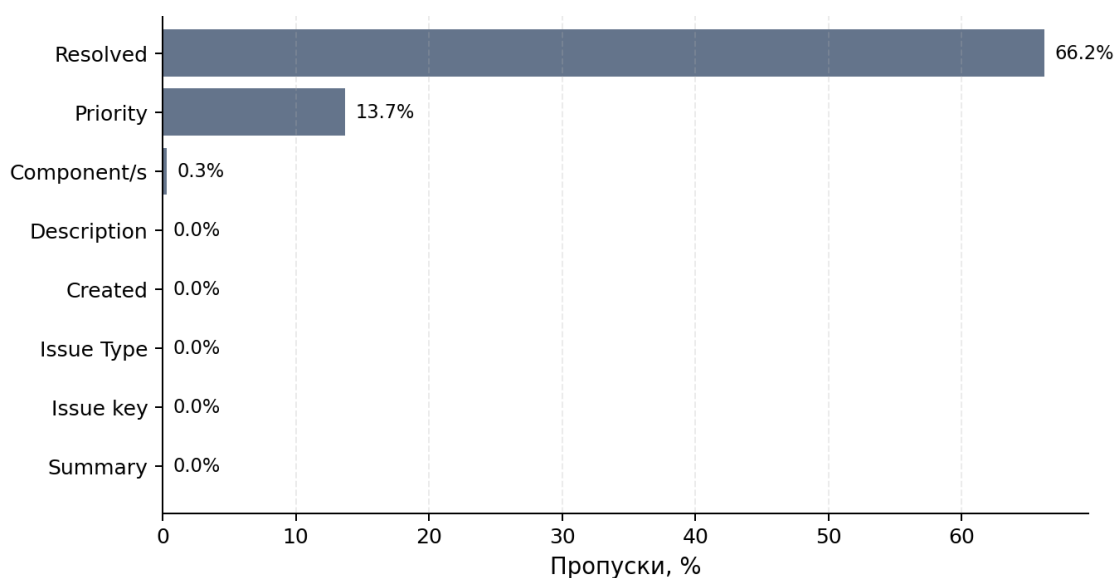


Рисунок 1.3 — Пропуски в ключевых полях исходного датасета

Распределение целевой переменной представлено на рисунке 1.1. По графику видно, что положительный класс `is_delayed = 1` встречается реже отрицательного класса, что подтверждает наличие умеренного дисбаланса. Распределение времени закрытия Jira issue представлено на рисунке 1.2 и демонстрирует длинный хвост. Пропуски в ключевых полях исходного датасета представлены на рисунке 1.3. Наиболее значимый пропуск связан с полем

Resolved, поэтому для обучения использовались только задачи с валидными датами Created и Resolved.

2 МОДЕЛИ И ЭКСПЕРИМЕНТЫ

2.1 Базовые модели

После подготовки данных была построена базовая модель, необходимая для получения отправной точки качества. В качестве baseline использовалась модель LogisticRegression. Выбор линейной модели в качестве базовой обусловлен тем, что она быстро обучается, хорошо воспроизводится, имеет понятную интерпретацию и позволяет оценить, насколько задача решается простыми методами без сложных ансамблевых алгоритмов.

Базовая модель обучалась на том же наборе признаков, что и последующие улучшенные модели. Это позволяет корректно сравнивать качество алгоритмов между собой, поскольку различие в результатах связано не с разными входными данными, а именно с особенностями моделей.

Для подготовки признаков использовался единый preprocessing pipeline. Числовые признаки обрабатывались с помощью заполнения пропусков и масштабирования, категориальные признаки — с помощью заполнения пропусков и one-hot encoding. Такой подход позволяет привести данные к формату, пригодному для обучения моделей scikit-learn, и одновременно избежать расхождения между обработкой данных на этапе обучения и на этапе предсказания.

По результатам эксперимента LogisticRegression показала следующие метрики: Accuracy = 0.5083, Precision = 0.3112, Recall = 0.8068, F1 = 0.4491, ROC-AUC = 0.6436. Полученные значения показывают, что базовая модель способна находить значительную часть задач с долгим закрытием, но при этом качество ранжирования и баланс между precision и recall остаются ограниченными. Поэтому baseline используется не как финальное решение, а как точка сравнения для более сложных моделей.

2.2 Улучшенные модели и эксперименты

На следующем этапе были обучены улучшенные модели: RandomForestClassifier и GradientBoostingClassifier. Обе модели относятся к ансамблевым методам на основе деревьев решений, однако используют разные подходы к построению итогового прогноза.

RandomForest строит набор независимых деревьев решений и объединяет их результаты. Такой подход часто хорошо работает на табличных данных, устойчив к нелинейным зависимостям между признаками и может учитывать взаимодействия между категориальными и числовыми признаками. Для данной задачи это важно, поскольку риск долгого закрытия issue может зависеть не от одного отдельного признака, а от их сочетания: например, типа задачи, наличия приоритета, компонента и объема текстового описания.

GradientBoosting также использует деревья решений, но строит их последовательно: каждая следующая модель пытается исправить ошибки предыдущих. Такой подход часто позволяет получать высокую точность, однако итоговое качество зависит от баланса между precision и recall, а также от выбранного порога классификации.

В качестве улучшения относительно baseline использовался не новый набор данных, а переход от простой линейной модели к нелинейным ансамблевым алгоритмам. Признаковое пространство оставалось одинаковым для всех моделей, что позволило сравнить именно влияние алгоритма на качество прогноза. Поверх baseline были протестированы RandomForestClassifier и GradientBoostingClassifier. Аугментации, нейросетевые архитектуры и сложный тюнинг гиперпараметров в текущей версии проекта не применялись, поскольку задача решалась на табличных данных, а основной целью было построить воспроизводимый end-to-end ML-сервис.

На практике переход к RandomForest дал существенное улучшение по F1, Recall и ROC-AUC относительно baseline. Это означает, что ансамбль деревьев лучше улавливает нелинейные зависимости между признаками. GradientBoosting

улучшил Accuracy и Precision, однако резко снизил Recall. Поэтому это изменение оказалось менее подходящим для прикладного сценария раннего предупреждения, несмотря на высокую точность срабатываний.

2.3 Экспериментальный протокол и сравнение моделей

Обучение реализовано в файле `project/src/train.py`. Конфигурация экспериментов находится в `project/configs/config.yaml`. Для воспроизводимости результатов используется фиксированный `random_seed = 42`. Данные разделяются на обучающую и тестовую выборки в пропорции 80/20 со stratify по целевой переменной. Стратификация необходима для того, чтобы сохранить примерно одинаковое соотношение классов в train и test выборках, так как положительный класс `is_delayed = 1` составляет около четверти наблюдений.

Итоговый sklearn Pipeline включает ColumnTransformer с SimpleImputer, StandardScaler, OneHotEncoder и классификатором. Финальная модель сохраняется в `project/models/model.joblib`, метаданные — в `project/models/model_metadata.json`, таблица метрик — в `project/artifacts/leaderboard.json`. Дополнительно для анализа экспериментов используется notebook `project/notebooks/02_modeling.ipynb`.

Таблица 2.1 — Сравнение моделей по целевым метрикам

Модель	Описание	Accuracy	Precision	Recall	F1	ROC-AUC	Комментарий
LogisticRegression	Baseline, линейная модель	0.5083	0.3112	0.8068	0.4491	0.6436	Для сравнения
RandomForest	Improved, ансамбль деревьев	0.6933	0.4465	0.9781	0.6131	0.9198	Финальная модель
GradientBoosting	Improved, boosting деревьев	0.7972	0.9364	0.1968	0.3253	0.8960	Высокий precision, низкий recall

По таблице видно, что модели демонстрируют разное поведение. LogisticRegression имеет достаточно высокий Recall = 0.8068 для базового

алгоритма, но при этом ROC-AUC = 0.6436 и F1 = 0.4491 показывают, что качество разделения классов остается умеренным. Это ожидаемо для простой линейной модели, поскольку признаки и целевая переменная могут иметь нелинейные зависимости.

RandomForest показывает лучший F1 = 0.6131, лучший Recall = 0.9781 и высокий ROC-AUC = 0.9198. Это означает, что модель почти не пропускает задачи, которые относятся к классу долгого закрытия, и при этом хорошо ранжирует issues по вероятности риска. Для прикладного сценария раннего предупреждения такое поведение является предпочтительным, поскольку пропуск действительно проблемной задачи может быть более критичным, чем лишнее предупреждение.

GradientBoosting показывает самую высокую Accuracy = 0.7972 и Precision = 0.9364. Однако Recall = 0.1968 является слишком низким для задачи раннего выявления рискованных issues. Такая модель делает мало ложных срабатываний, но пропускает большую часть задач, которые действительно будут закрываться долго. Для бизнес-сценария мониторинга и предупреждения это является существенным недостатком.

Таким образом, один только показатель Accuracy не является достаточным для выбора модели. В условиях дисбаланса классов и прикладной задачи раннего выявления важнее учитывать Recall, F1 и ROC-AUC. Модель с высокой accuracy, но низким recall может выглядеть качественной формально, однако быть непригодной для практического использования, если она пропускает большинство рискованных задач.

Эксперименты проводились по единому протоколу. Сначала исходные данные загружались из project/data/jira_dataset.csv, после чего выполнялась фильтрация строк с валидными датами Created и Resolved. Далее рассчитывалось время закрытия resolution_time_days и формировалась целевая переменная is_delayed на основе 75-го перцентиля.

После формирования целевой переменной из данных удалялись признаки, которые могли привести к утечке информации: Status, Resolution, Resolved,

Updated и resolution_time_days. Эти поля не должны использоваться при прогнозировании, так как они содержат сведения, известные после закрытия или обновления задачи. Использование таких признаков привело бы к завышенным метрикам и сделало бы модель неприменимой в реальном сценарии ранней оценки issue.

Финальный набор признаков включал issue_type, priority, has_priority, component_present, summary_length, summary_word_count, description_length и description_word_count. Эти признаки доступны на раннем этапе обработки задачи и не требуют информации о ее будущем состоянии.

Для всех моделей использовалось одинаковое разбиение train/test = 80/20 и фиксированный random_seed = 42. Метрики рассчитывались на тестовой выборке, которая не использовалась при обучении. Это позволяет оценить способность модели обобщать закономерности на новых данных.

При сравнении моделей учитывались следующие показатели:

Ассигасу показывает общую долю правильных ответов, однако в данной задаче не может быть основной метрикой из-за дисбаланса классов.

Precision показывает, какая доля задач, предсказанных как долгие, действительно относится к классу долгого закрытия.

Recall показывает, какую долю всех реально долгих задач модель смогла обнаружить.

F1 объединяет precision и recall в одну метрику и помогает оценить баланс между ложными предупреждениями и пропущенными рискованными задачами.

ROC-AUC показывает качество ранжирования задач по вероятности принадлежности к положительному классу.

Такой набор метрик позволяет оценивать модели не только формально, но и с учетом прикладного сценария. Для сервиса раннего предупреждения особенно важны Recall и F1, поскольку модель должна помогать команде находить потенциально проблемные issues до того, как они станут критичными.

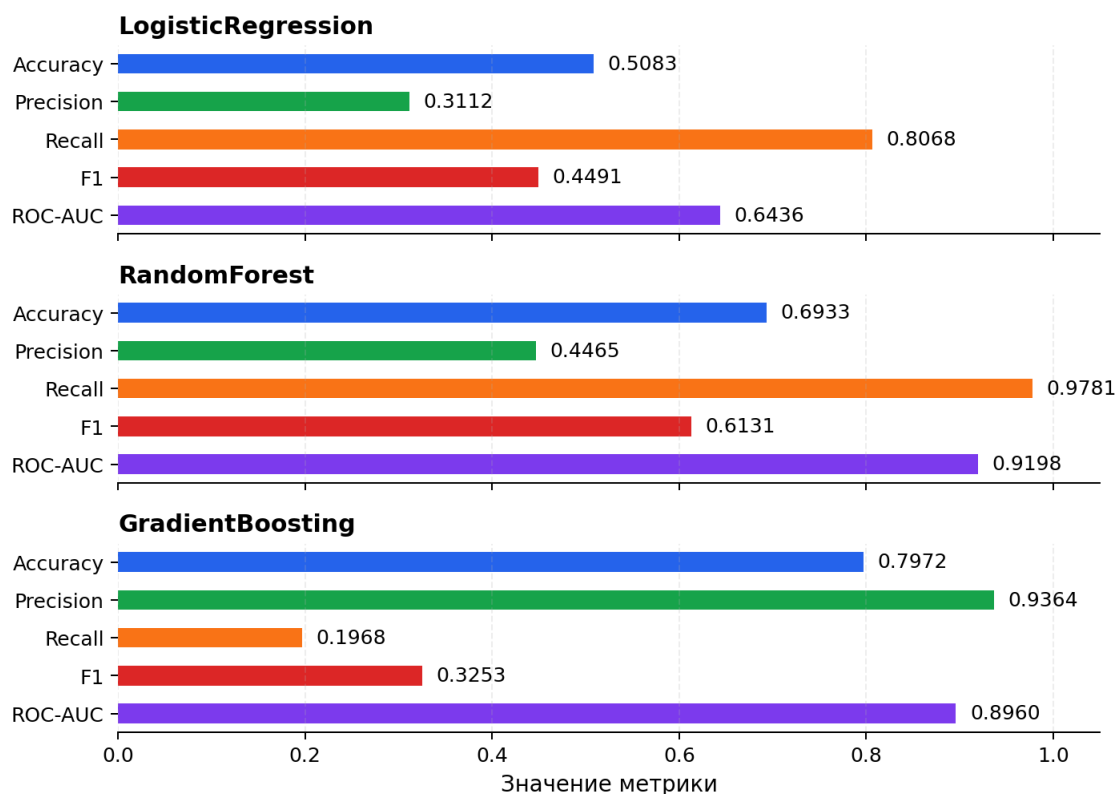


Рисунок 2.1 — сравнение метрик моделей

2.4 Выбор финальной модели

Финальной моделью выбрана `RandomForestClassifier`. Она показала лучший баланс качества среди протестированных моделей: $F1 = 0.6131$, $Recall = 0.9781$ и $ROC-AUC = 0.9198$. Эти значения указывают, что модель хорошо выявляет задачи с риском долгого закрытия и при этом уверенно ранжирует `issues` по вероятности риска.

Главное преимущество `RandomForest` в данном проекте — высокий `Recall`. Для задачи раннего выявления долгих `issues` это особенно важно, потому что пропущенная рискованная задача может привести к задержкам, накоплению нерешенных обращений и ухудшению управляемости процесса. Ложное предупреждение в таком сценарии менее критично: `project manager` или тимлид может дополнительно проверить задачу и принять решение вручную.

При этом у RandomForest Precision = 0.4465, что указывает на наличие ложных срабатываний. Это является ограничением модели, однако в контексте проекта такой компромисс допустим. Сервис не предназначен для автоматического принятия окончательных управленческих решений; он должен работать как вспомогательный инструмент, который подсвечивает задачи, требующие дополнительного внимания.

GradientBoosting не был выбран финальной моделью, несмотря на высокие Accuracy = 0.7972 и Precision = 0.9364. Основная причина — низкий Recall = 0.1968. Такая модель слишком редко относит задачи к классу долгого закрытия и, следовательно, пропускает большую часть реально рискованных issues. Для сценария раннего предупреждения это делает ее менее полезной, чем RandomForest.

LogisticRegression также не была выбрана финальной моделью, поскольку уступает RandomForest по F1 и ROC-AUC. При этом она остается полезной как baseline, позволяющий показать, что переход к ансамблевой модели действительно улучшает качество решения.

При выборе финальной модели учитывались не только метрики качества, но и прикладные trade-off. RandomForest сложнее LogisticRegression и требует больше ресурсов, однако в рамках текущего объема данных остается достаточно легкой моделью для локального запуска и API-сервиса. По сравнению с GradientBoosting финальная модель дает больше ложных предупреждений, но значительно реже пропускает действительно долгие issues. Для сценария раннего выявления это более приемлемый компромисс: лишняя проверка задачи менее критична, чем пропуск задачи, которая затем будет закрываться очень долго.

Как показано в таблице 2.1, RandomForest имеет лучший F1 = 0.6131, лучший Recall = 0.9781 и высокий ROC-AUC = 0.9198. Поэтому RandomForestClassifier выбран как наиболее подходящий компромисс между качеством прогнозирования, устойчивостью на табличных данных, скоростью работы и прикладными требованиями задачи. Модель сохраняется в

`project/models/model.joblib` и используется в FastAPI-сервисе для обработки запросов к эндпоинту `/predict`.

3 АРХИТЕКТУРА РЕШЕНИЯ И СЕРВИС

3.1 Архитектура пайплайна

Архитектура решения строится по цепочке: загрузка данных → разведочный анализ → предобработка → обучение модели → сохранение модели → запуск сервиса → получение предсказаний через API. Такая структура позволяет отделить исследовательскую часть проекта от сервисной части. Notebook используется для анализа и визуализации, `src/train.py` — для обучения, `models/model.joblib` — для хранения финальной модели, а `src/service.py` — для предоставления модели через HTTP API.

На первом этапе исходные данные загружаются из файла `project/data/jira_dataset.csv`. Далее выполняется разведочный анализ данных в notebook `project/notebooks/01_eda.ipynb`. В ходе анализа проверяются пропуски, типы признаков, распределение целевой переменной и особенности времени закрытия задач. После этого данные проходят предобработку: парсятся даты `Created` и `Resolved`, рассчитывается `resolution_time_days`, формируется бинарная целевая переменная `is_delayed` и создаются признаки, доступные на этапе первичной triage-оценки `issue`.

Предобработка вынесена в отдельный модуль `project/src/preprocessing.py`. Это важно с инженерной точки зрения, поскольку один и тот же `preprocessing pipeline` используется как при обучении модели, так и при последующем предсказании. Такой подход снижает риск ситуации, когда модель обучалась на данных, обработанных одним способом, а в сервисе получает признаки в другом формате.

Обучение моделей реализовано в `project/src/train.py`. В рамках экспериментов сравниваются базовая модель `LogisticRegression` и улучшенные модели `RandomForestClassifier` и `GradientBoostingClassifier`. После сравнения по метрикам финальной моделью выбрана `RandomForestClassifier`. Обученная

модель сохраняется в `project/models/model.joblib`, а ее метаданные — в `project/models/model_metadata.json`. Сохранение модели как отдельного артефакта позволяет сервису не обучать модель заново при каждом запуске, а загружать уже подготовленную версию.

API-сервис реализован в `project/src/service.py` на базе FastAPI. При запуске сервис загружает сохраненную модель и метаданные, после чего становится доступен для HTTP-запросов. Пользователь или внешняя система может отправить описание одной или нескольких Jira issues в формате JSON на эндпоинт `/predict` и получить прогноз риска долгого закрытия.

Таким образом, архитектура проекта разделяет этапы обучения и эксплуатации. Обучение выполняется отдельно и формирует артефакты модели, а сервис отвечает только за загрузку модели, валидацию входных данных, применение предобработки и выдачу прогноза. Это делает решение более воспроизводимым, понятным и удобным для демонстрации.



Рисунок 3.1 — Схема архитектуры решения

3.2 API и эндпоинты

Сервис реализован в `project/src/service.py` на FastAPI. В проекте реализованы прикладные эндпоинты GET `/health`, POST `/predict` и GET `/metrics`. Swagger UI не является отдельной бизнес-функцией проекта, а автоматически формируется FastAPI и доступен по адресу <http://localhost:8000/docs>. Скриншот Swagger UI сохранен в `project/screenshots/swagger_ui.png` и подтверждает работоспособность сервиса.

`/health` возвращает `status`, `environment`, `model_loaded`, `model_name`, `model_version` и путь к датасету. `/predict` принимает JSON со списком `issues` и возвращает `model_name`, `model_version` и массив `predictions`. Для каждой `issue` возвращаются `prediction`, `probability`, `is_delayed`, `delay_probability` и `delay_risk`.

Статус-коды: 200 — успешный прогноз; 422 — ошибка Pydantic-валидации входного JSON; 400 — некорректные значения при подготовке данных; 500 — внутренняя ошибка предсказания или недоступность модели

Листинг 1 — пример запроса и ответа эндпоинта `/predict`

```
$ curl -X POST "http://localhost:8000/predict" ^  
-H "Content-Type: application/json" ^  
--data @src/demo_request.json  
  
{  
  "model_name": "random_forest",  
  "model_version": "v1",  
  "predictions": [  
    {  
      "prediction": 0,  
      "probability": 0.1699,  
      "is_delayed": 0,  
      "delay_probability": 0.1699,  
      "delay_risk": "low"  
    },  
    {  
      "prediction": 0,  
      "probability": 0.3927,  
      "is_delayed": 0,  
      "delay_probability": 0.3927,  
      "delay_risk": "medium"  
    }  
  ]  
}
```

В сервисе предусмотрена обработка типовых ситуаций. При корректном запросе возвращается статус 200. Если входной JSON не соответствует Pydantic-схеме, возвращается ошибка 422. Если данные имеют некорректные значения на этапе подготовки признаков, может возвращаться ошибка 400. Если модель недоступна или во время предсказания возникает внутренняя ошибка, возвращается статус 500.

Использование Pydantic-схем в `project/src/schemas.py` делает API более надежным. Схемы явно описывают структуру входного запроса и выходного ответа, а также позволяют автоматически документировать сервис в Swagger UI. Это снижает вероятность некорректного использования API и упрощает проверку проекта на защите.

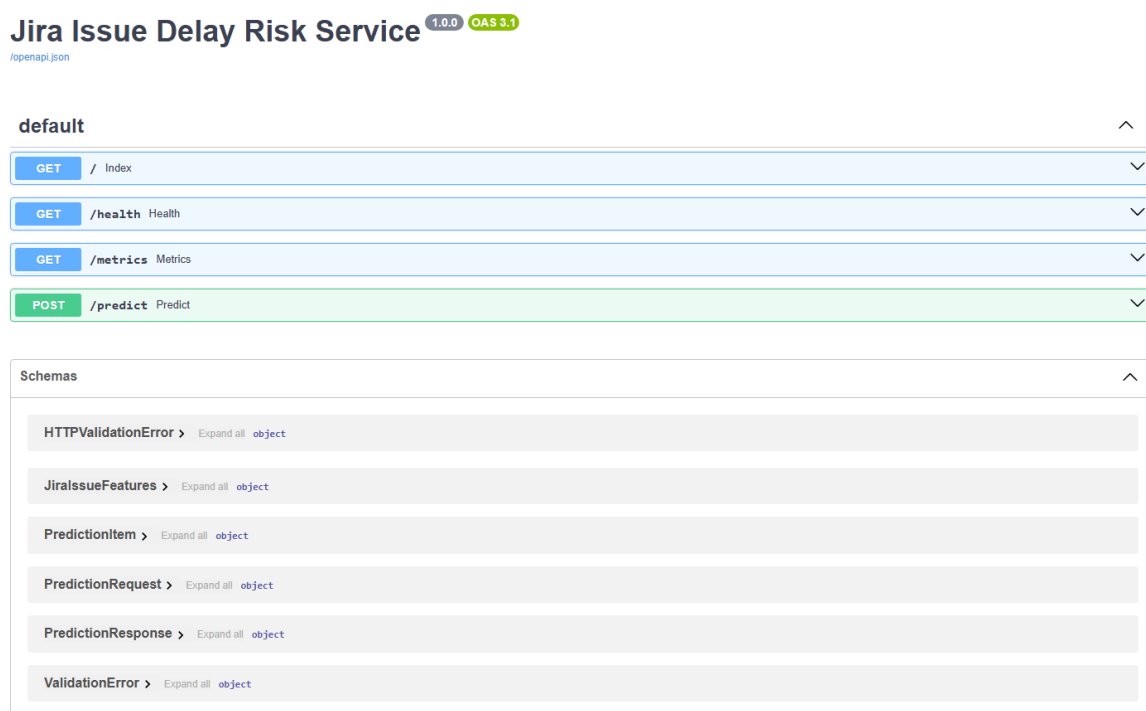


Рисунок 3.2 — Интерфейс Swagger UI работающего сервиса (подтверждение работоспособности)

3.3 Технологический стек и контейнеризация

В проекте используется технологический стек, ориентированный на воспроизводимость, простоту демонстрации и совместимость с типовыми ML-задачами на табличных данных. Основной язык разработки — Python. Для работы с данными используется `pandas`, для визуализации в notebook — `matplotlib`, для обучения моделей и построения preprocessing pipeline — `scikit-learn`, для сохранения модели — `joblib`.

Сервисная часть реализована на FastAPI. Для запуска ASGI-приложения используется `Uvicorn`. `Pydantic` применяется для описания и проверки схем входных и выходных данных. Такой стек позволяет быстро развернуть ML-модель как HTTP-сервис и предоставить пользователю понятный интерфейс взаимодействия через REST API и Swagger UI.

Зависимости проекта зафиксированы в `project/requirements.txt`. Это позволяет воспроизвести окружение на другой машине и снизить риск ошибок, связанных с отсутствующими библиотеками. Основные параметры проекта вынесены в конфигурационные файлы и переменные окружения, что позволяет менять пути к модели, версию модели, порт сервиса и другие настройки без изменения основного кода.

Для контейнеризации подготовлен `Dockerfile`, расположенный в `project/Dockerfile`. Контейнеризация нужна для того, чтобы сервис можно было запускать в изолированном и воспроизводимом окружении. Это особенно важно для ML-проектов, где результат зависит не только от кода, но и от версий библиотек, структуры файлов и наличия артефактов модели.

Листинг 2 — сборка и запуск сервиса

```
$ docker build -t aie-project .
$ docker run -p 8000:8000 aie-project
# альтернативно через compose
$ docker compose up --build
# без Docker
$ pip install -r requirements.txt
$ uvicorn src.service:app --host 0.0.0.0 --port 8000
```

Для проверки основного сценария работы используется запрос к /predict с демонстрационным JSON из файла src/demo_request.json. После этого можно открыть Swagger UI по адресу <http://localhost:8000/docs> и убедиться, что сервис содержит эндпоинты /health, /predict и /metrics, а также схемы входных и выходных данных.

4 НАБЛЮДАЕМОСТЬ, КОНФИГУРАЦИЯ И БЕЗОПАСНОСТЬ

4.1 Логирование и наблюдаемость

Для эксплуатации ML-сервиса важно не только получить корректную модель, но и иметь возможность контролировать состояние приложения во время работы. Даже если модель показывает приемлемые метрики на тестовой выборке, в сервисном режиме могут возникать ошибки загрузки артефактов, некорректные входные данные, проблемы с форматом запроса или внутренние исключения при предсказании. Поэтому в проекте реализованы базовые механизмы наблюдаемости.

В сервисе логируются ключевые события жизненного цикла приложения: запуск сервиса, загрузка модели, обращения к API, ошибки валидации, ошибки предсказания и время обработки запросов. Это позволяет понять, корректно ли стартовало приложение, была ли успешно загружена модель и как сервис реагирует на входящие запросы.

Логирование особенно важно для эндпоинта `/predict`, так как именно он выполняет основную бизнес-функцию сервиса. При обращении к `/predict` в логах фиксируется факт обработки запроса, количество переданных `issues` и время выполнения. Если запрос имеет неправильную структуру или содержит некорректные значения, соответствующее предупреждение или ошибка также попадает в лог. Благодаря этому можно быстрее найти причину проблемы: ошибка в формате JSON, ошибка валидации Pydantic, недоступность модели или внутренняя ошибка предобработки.

Пример логов работающего сервиса:

Листинг 3 — пример лога запросов сервиса

```
2026-06-18 18:39:23,053 | INFO | src.service | Model loaded from  
...\project\models\model.joblib  
2026-06-18 18:39:23,053 | INFO | src.service | Service started with  
model=random_forest version=v1  
2026-06-18 18:39:23,055 | INFO | src.service | GET /health -> 200 in  
1.01 ms  
2026-06-18 18:39:23,057 | WARNING | src.service | POST /predict  
validation failed: ...  
2026-06-18 18:39:23,069 | INFO | src.service | /predict scored 1  
issue(s)
```

Для быстрой проверки состояния приложения реализован эндпоинт GET /health. Он возвращает статус сервиса, информацию об окружении, признак загрузки модели, имя модели, версию модели и путь к датасету. Такой эндпоинт полезен при локальном запуске, демонстрации проекта и контейнерном развертывании, так как позволяет быстро убедиться, что сервис доступен и готов принимать запросы.

По логам корректная работа сервиса определяется следующим образом: при запуске должна появиться запись об успешной загрузке модели; при обращении к /health должен фиксироваться статус 200; при обращении к /predict должно логироваться количество обработанных issues и время выполнения запроса; ошибки валидации должны фиксироваться как warning или error, но не приводить к аварийному завершению сервиса. Если модель не загружена или запрос не может быть обработан, соответствующая причина должна быть отражена в логах.

Дополнительно реализован эндпоинт GET /metrics. Он возвращает базовые технические метрики работы сервиса: количество обработанных запросов, количество предсказанных issues, число ошибок, время работы приложения и latency последнего запроса. В текущей версии эти метрики являются простыми и предназначены прежде всего для демонстрации наблюдаемости. При дальнейшем развитии проекта их можно расширить и интегрировать с полноценными системами мониторинга.

В текущей версии проекта полноценный мониторинг дрейфа данных и качества модели в эксплуатации не реализован. Однако наличие /metrics и логирования создает основу для дальнейшего расширения наблюдаемости,

например добавления счетчиков распределения входных признаков, мониторинга вероятностей и контроля изменения качества модели на новых данных.

Наличие `/health`, `/metrics` и логирования делает сервис более прозрачным. Пользователь или разработчик может не только получить прогноз, но и проверить, что модель действительно загружена, приложение работает стабильно, ошибки фиксируются, а поведение сервиса можно отследить по журналам.

4.2 Конфигурация

Параметры проекта вынесены из кода в конфигурационные файлы и переменные окружения. Это делает проект более гибким и удобным для воспроизведения, так как основные настройки можно менять без изменения исходного кода.

Основной конфигурационный файл расположен по пути `project/configs/config.yaml`. В нем описаны пути к данным и артефактам, параметры обучения, фиксированный `random seed`, используемые признаки, метрики и параметры моделей. Наличие единого конфигурационного файла позволяет централизованно управлять экспериментами и снижает риск появления несогласованных настроек в разных частях проекта.

Работа с конфигурацией вынесена в модуль `project/src/config.py`. Такой подход отделяет бизнес-логику и ML-логику от кода загрузки настроек. В результате сервис, обучение и вспомогательные скрипты могут использовать общую конфигурацию, не дублируя пути и параметры.

Пример переменных окружения находится в файле `project/.env.example`. Он содержит шаблон локальной настройки окружения и не включает реальные секреты.

В `configs/config.yaml` и `.env.example` вынесены параметры, которые можно менять без изменения кода: пути к данным и модели, путь к метаданным модели, версия модели, `host` и `port` сервиса, уровень логирования, `random seed`, список признаков, параметры разбиения выборки и настройки моделей. Это позволяет использовать один и тот же код в разных режимах запуска: локальное обучение, локальный сервис, контейнерный запуск и демонстрация проекта.

Использование конфигурации особенно важно для сервисной части проекта. Например, путь к модели, версия модели или уровень логирования могут отличаться в локальном режиме, при демонстрации и при контейнерном запуске. Если эти значения вынесены в конфиг и переменные окружения, проект проще адаптировать под разные условия запуска.

Таким образом, конфигурационный слой проекта решает несколько задач: повышает воспроизводимость, упрощает изменение параметров, уменьшает количество жестко заданных значений в коде и делает структуру проекта более понятной для проверки.

4.3 Работа с секретами и данными

В репозитории не хранятся реальные токены, пароли, приватные ключи и персональные данные. Для локальных настроек используется `project/.env.example`, а реальный `.env` исключен через `.gitignore`. В логах не выводятся секреты, пароли, токены и полное содержимое пользовательских данных: фиксируются только технические события, статусы запросов, ошибки и время обработки.

В проекте предусмотрены базовые меры безопасной работы с секретами и данными. Несмотря на то что текущая версия сервиса не использует внешние API-токены, пароли или закрытые учетные данные, структура проекта подготовлена таким образом, чтобы реальные секреты не попадали в репозиторий.

Для этого используется файл `project/.env.example`. Он содержит только пример переменных окружения и служит шаблоном для создания локального `.env`. Реальный `.env` не должен коммититься в репозиторий. Это важно, потому что в реальных проектах в таком файле могут храниться токены доступа, пароли, адреса внутренних сервисов или другие чувствительные параметры.

Файл `project/.gitignore` исключает `.env`, кеш, временные файлы и большие CSV-файлы в папке `data/`. Исключение исходного CSV из публичного репозитория также связано с тем, что данные получены с Kaggle, и перед публикацией необходимо учитывать условия использования датасета. Поэтому в `project/data/README.md` находится инструкция по получению данных, а не обязательно сам исходный файл.

Отдельно учитывается безопасность логирования. В логах не должны выводиться секреты, токены, пароли и персональные данные. В текущей версии сервиса логируются технические события: запуск приложения, загрузка модели, статусы запросов, ошибки и `latency`. Такой подход позволяет отслеживать работу сервиса без раскрытия чувствительной информации.

Также при построении модели была учтена методологическая безопасность данных — предотвращение утечки целевой переменной. В признаки не включаются поля `Status`, `Resolution`, `Resolved`, `Updated` и `resolution_time_days`, так как они содержат информацию, недоступную на этапе раннего прогнозирования. Это относится не к информационной безопасности в узком смысле, а к корректности ML-эксперимента: модель не должна получать будущую информацию, иначе метрики будут нечестно завышены.

Таким образом, в проекте реализованы базовые практики безопасной организации репозитория: используется `.env.example` вместо реального `.env`, чувствительные файлы исключены через `.gitignore`, секреты не коммитятся, в логах не выводятся приватные данные, а признаки модели выбраны с учетом предотвращения утечки данных.

ЗАКЛЮЧЕНИЕ

В ходе выполнения итогового проекта был разработан воспроизводимый ML-сервис для прогнозирования риска долгого закрытия задач в Jira. Проект охватывает основные этапы жизненного цикла инженерии искусственного интеллекта: анализ данных, формирование целевой переменной, подготовку признаков, обучение и сравнение моделей, выбор финального алгоритма, сохранение модели, реализацию API-сервиса, логирование, конфигурацию, тестирование и подготовку контейнерного запуска.

В качестве исходных данных использовался открытый Jira Dataset [1]. После фильтрации задач с валидными датами Created и Resolved для обучения осталось 16562 решенных issues. Целевая переменная is_delayed была сформирована на основе времени закрытия задачи: issue относилась к классу долгого закрытия, если resolution_time_days превышал 75-й перцентиль. В текущей версии проекта этот порог составил 305.2056 дня.

В рамках экспериментов были сравнены LogisticRegression, RandomForestClassifier и GradientBoostingClassifier. Финальной моделью выбрана RandomForestClassifier, так как она показала лучший $F1 = 0.6131$, лучший $Recall = 0.9781$ и высокий $ROC-AUC = 0.9198$. Такой выбор соответствует прикладному сценарию раннего предупреждения: для project manager, тимлида или команды поддержки важнее не пропустить потенциально долгую задачу, чем получить максимальную Accuracy за счет пропуска положительного класса.

Инженерная часть проекта реализована в виде FastAPI-сервиса. Сервис содержит эндпоинты GET /health, POST /predict и GET /metrics. Swagger UI доступен по адресу <http://localhost:8000/docs>, а скриншот работающего интерфейса сохранен в project/screenshots/swagger_ui.png. Предобработка вынесена в project/src/preprocessing.py, обучение — в project/src/train.py, сервис

— в `project/src/service.py`, схемы API — в `project/src/schemas.py`. Финальная модель сохраняется в `project/models/model.joblib`.

Для воспроизводимости подготовлены `requirements.txt`, `configs/config.yaml`, `.env.example`, `Dockerfile`, `docker-compose.yml` и `README`. В конфигурацию вынесены пути к данным и модели, параметры обучения, `random seed`, признаки, метрики и параметры сервиса. Реальный `.env` не коммитится, секреты и персональные данные в логах не выводятся. Работоспособность API проверяется тестами из `project/tests/test_service.py`.

Текущая версия проекта имеет ограничения. Модель обучена на данных одного Jira-проекта `SRCTREEWIN`, поэтому перед применением к другим проектам требуется дополнительная проверка. Порог 305.2056 дня основан на 75-м процентиле конкретного датасета и не является универсальным SLA. Текстовые поля `summary` и `description` используются только через простые признаки длины и количества слов, без TF-IDF, `embeddings` или специализированных NLP-моделей. Кроме того, обучение проводится только на решенных `issues`, что может создавать смещение относительно всех задач в Jira.

В качестве дальнейшего развития можно рассмотреть использование TF-IDF или `embeddings` для текстовых признаков, подбор оптимального порога классификации, калибровку вероятностей, `time-based validation`, проверку модели на других Jira-проектах, расширение мониторинга качества модели и интеграцию с реальным Jira API.

Сценарий демонстрации проекта на защите состоит из двух основных частей. Сначала запускается сервис локально через Uvicorn или, при наличии Docker, через контейнерный запуск. После запуска проверяется эндпоинт `/health`, который должен вернуть `status=ok` и `model_loaded=true`. Затем открывается Swagger UI по адресу <http://localhost:8000/docs> и выполняется запрос к `/predict` на примере `project/src/demo_request.json`. Основной демонстрационный сценарий — прогноз риска долгого закрытия для одной или нескольких Jira `issues`.

Дополнительно можно показать обработку некорректного запроса, при котором сервис возвращает ошибку валидации, а ошибка фиксируется в логах.

Таким образом, в рамках итогового проекта был разработан не только ML-эксперимент, но и минимальный сервис машинного обучения, демонстрирующий связь между анализом данных, построением модели и инженерной реализацией API.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Jira Dataset / Cesar Anasco.* — URL: <https://www.kaggle.com/datasets/cesaranasco/jira-dataset> (дата обращения: 17.06.2026).
2. *FastAPI Documentation.* — URL: <https://fastapi.tiangolo.com/> (дата обращения: 18.06.2026).
3. *scikit-learn User Guide.* — URL: https://scikit-learn.org/stable/user_guide.html (дата обращения: 18.06.2026).
4. *pandas Documentation.* — URL: <https://pandas.pydata.org/docs/> (дата обращения: 18.06.2026).
5. *Docker Documentation.* — URL: <https://docs.docker.com/> (дата обращения: 18.06.2026).
6. *Uvicorn Documentation.* — URL: <https://www.uvicorn.org/> (дата обращения: 18.06.2026).
7. *Pydantic Documentation.* — URL: <https://docs.pydantic.dev/> (дата обращения: 18.06.2026).

ПРИЛОЖЕНИЕ А

Чек-лист самопроверки проекта

Таблица А.1 — Чек-лист самопроверки

№	Критерий	Да/Нет	Где смотреть
1	Сервис запускается по инструкции из README и работает	Да	project/README.md, project/src/service.py
2	/predict использует реальную модель, а не заглушку	Да	project/src/service.py, project/models/model.joblib
3	Есть EDA и хотя бы один эксперимент с метриками	Да	project/notebooks/01_eda.ipynb, project/notebooks/02_modeling.ipynb
4	Есть baseline и улучшенная модель, сравнение по метрикам	Да	project/src/train.py, project/artifacts/leaderboard.json
5	Код структурирован в src/, а не свален в один ноутбук	Да	project/src/
6	Есть Dockerfile или внятный сценарий развёртывания	Да	project/Dockerfile, project/docker-compose.yml, project/README.md
7	Есть .env.example, нет реальных секретов в репозитории	Да	project/.env.example, project/.gitignore
8	Реализованы логи и эндпоинт /health	Да	project/src/service.py
9	Обоснован выбор финальной модели	Да	Раздел 2.4 отчета, project/artifacts/leaderboard.json
10	README и отчёт позволяют понять сценарий демонстрации	Да	project/README.md, заключение отчета